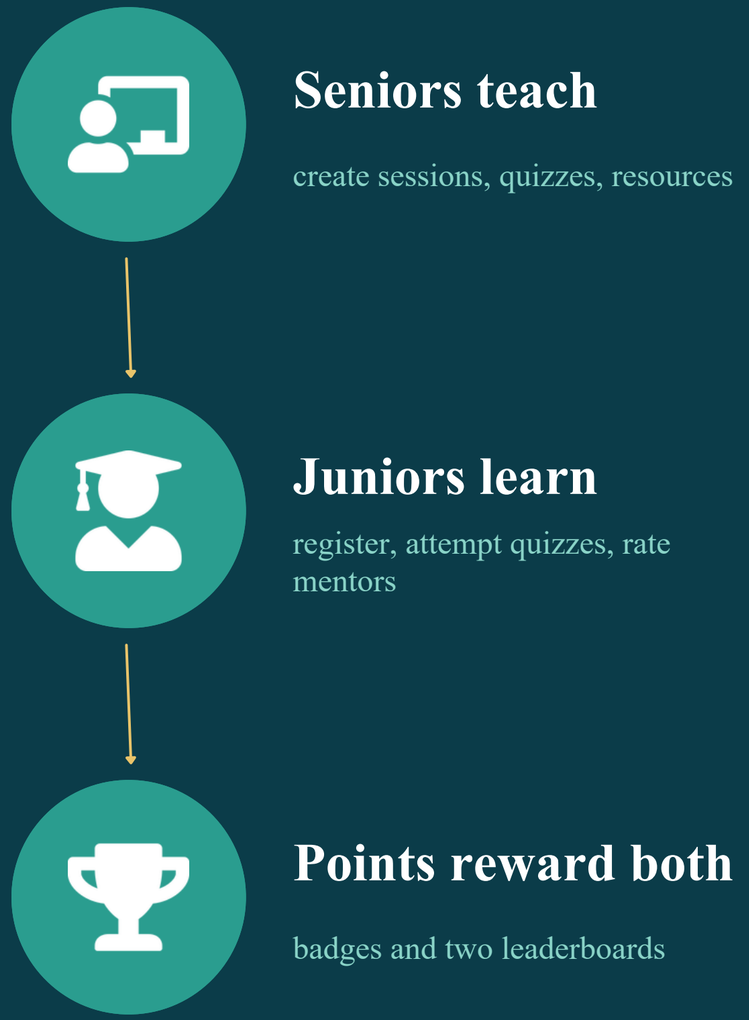
UIT3561---Full Stack Web Development

PeerBridge

Intercollege Peer Learning & Mentorship Platform

Project Presentation: Design, Implementation, Exception Handling

TEAM :
Roja Sri - 3122245002082
Rovena - 3122245002083
Sharal Jospheen rani - 3122245002099



Problem & Solution

The problem

- Juniors (Year 1-2) lack structured guidance for hard subjects and placements
- WhatsApp groups are untraceable and disappear quickly
- Paid platforms are generic, not aligned to the college syllabus or culture

The PeerBridge solution

- Seniors (Year 3-4) create sessions, quizzes and share resources
- Juniors discover, register, attempt quizzes and rate mentors
- Points, badges and leaderboards make participation measurable

6
modules

2
roles: junior, senior

8
MongoDB collections

21
designed test cases

Tech Stack & Project Structure

Frontend

React · Vite · JavaScript · React Router · Axios · CSS

Backend

Node.js · Express.js · JWT · bcryptjs · CORS · dotenv · nodemon

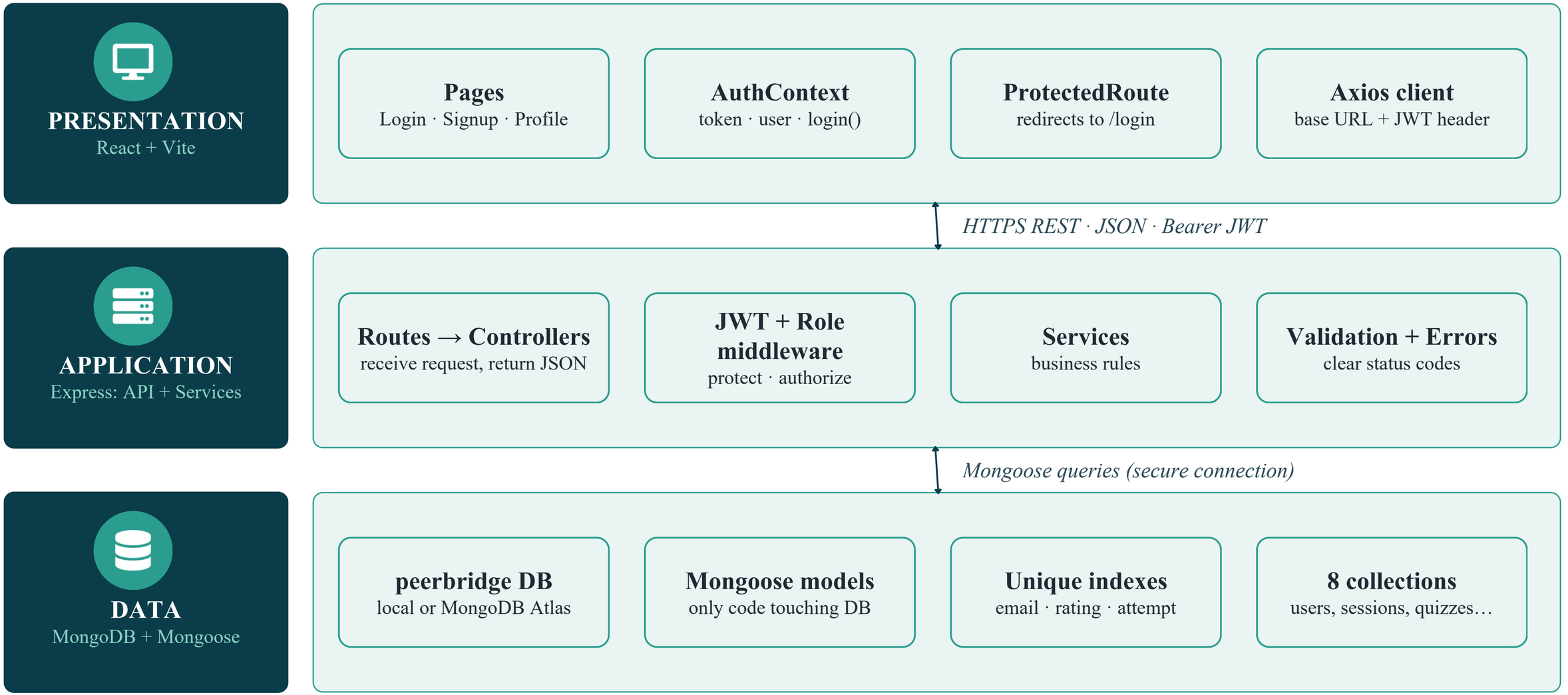
Database & Tools

MongoDB · Mongoose · Compass · VS Code · Git/GitHub · Postman

Current project structure

```
PeerBridge/
├── backend/
│   ├── config/db.js
│   ├── controllers/authController.js
│   ├── middleware/auth.js
│   ├── models/User.js
│   ├── routes/authRoutes.js
│   └── server.js  (.env ignored by Git)
└── frontend/src/
    ├── api/axios.js
    ├── components/ProtectedRoute.jsx
    ├── context/AuthContext.jsx
    ├── pages/ Login · Signup · Profile
    └── App.jsx · main.jsx
```

Three-Layer Architecture



Functionality of Each Layer



Presentation

React + Vite

- Signup, Login and Profile pages talk to the API
- AuthContext keeps token and user (login, logout, isLoggedIn)
- ProtectedRoute sends logged-out users to /login
- Axios attaches the Bearer token to every request

Login.jsx · Signup.jsx · Profile.jsx · AuthContext.jsx



Application

Express routes, middleware, services

- Routes map each URL to a controller
- protect verifies the JWT; authorize checks the role (403)
- Services hold the rules: seat booking, scoring, points
- Error handler turns failures into clean status codes

authRoutes.js · authController.js · middleware/auth.js



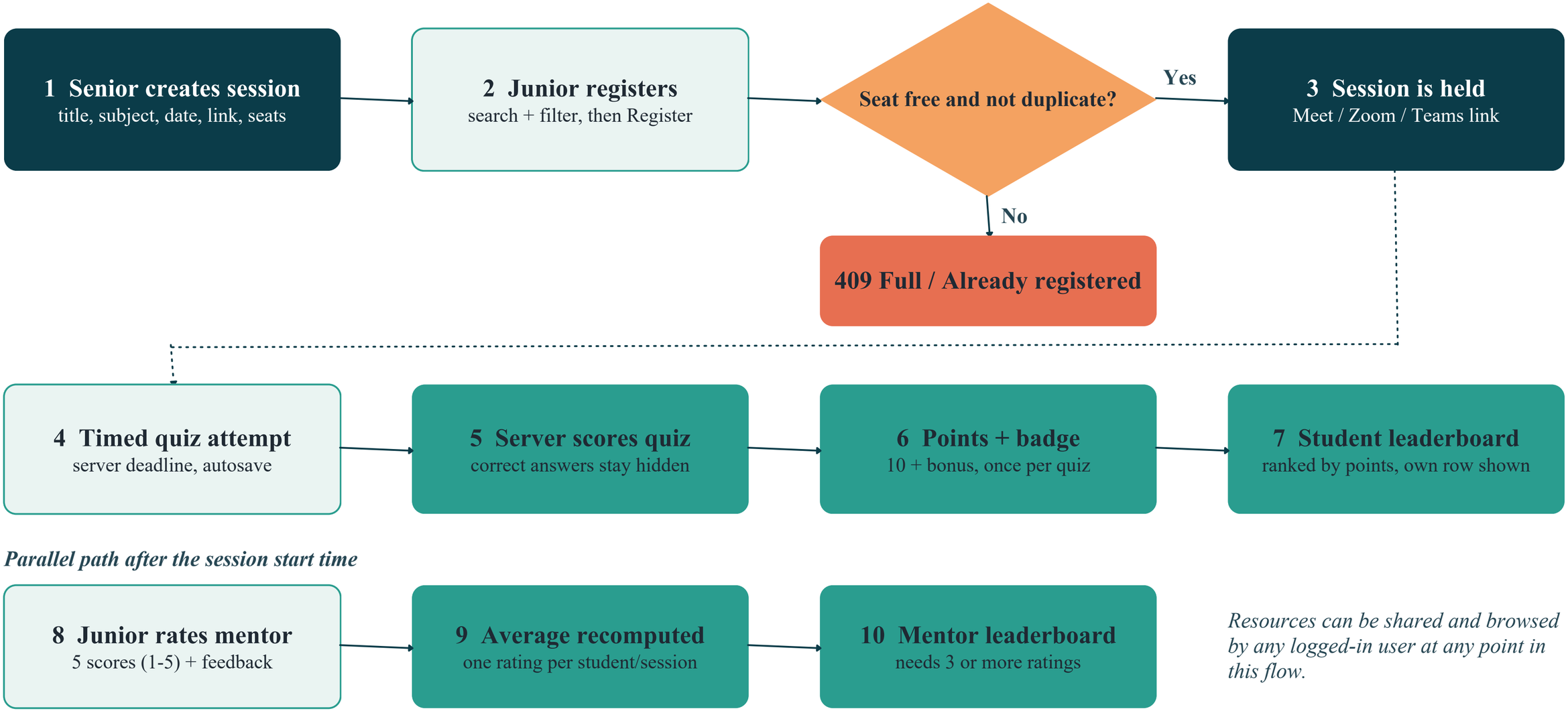
Data

MongoDB + Mongoose

- Models are the only code that touches the database
- Unique indexes enforce rules: email, rating, attempt
- point_transactions keeps an audit trail of points
- Atlas gives automatic cloud backup

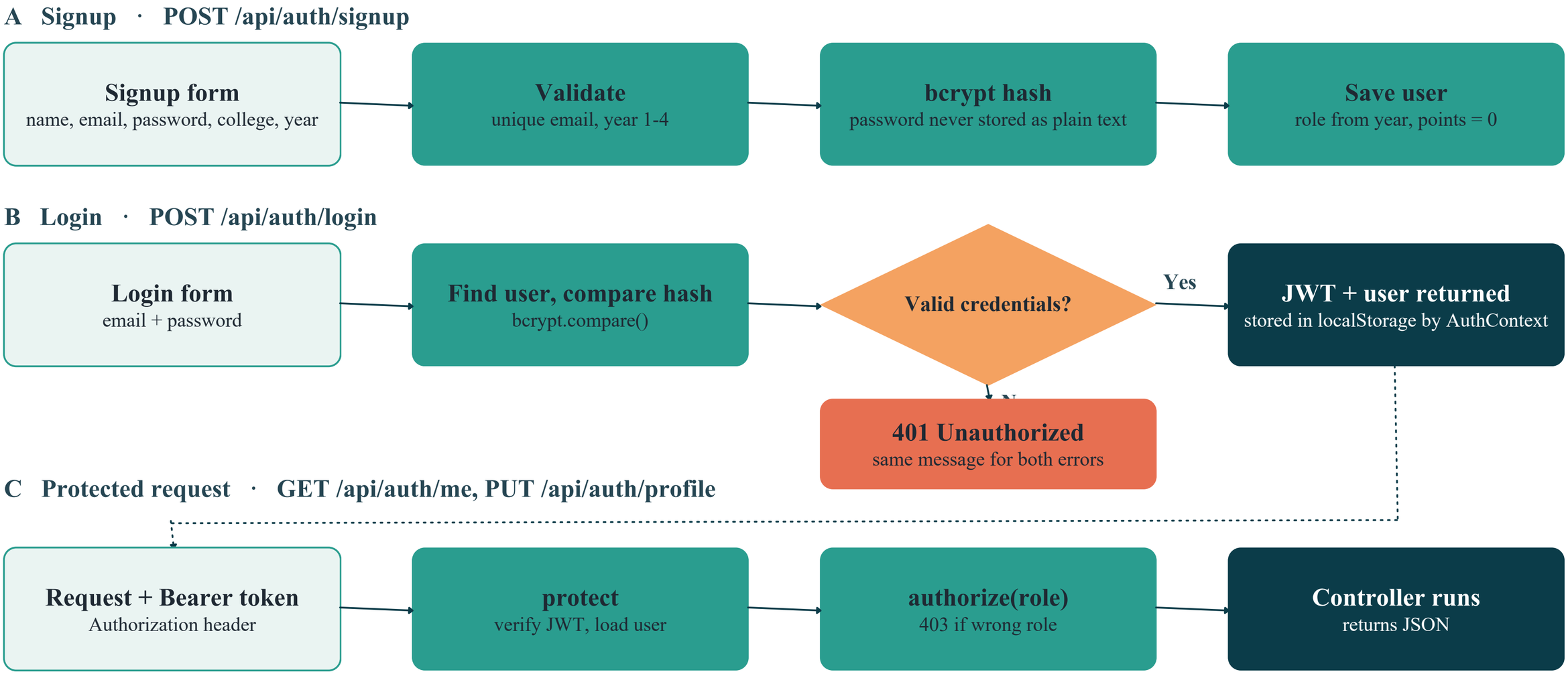
User.js · db.js · peerbridge database

Use Case: Senior-to-Junior Journey



Resources can be shared and browsed by any logged-in user at any point in this flow.

Authentication Flow (Member 1)



Code: Auth Middleware & Role Check

backend/middleware/auth.js (representative implementation)

```
const jwt  = require('jsonwebtoken');
const User = require('../models/User');

// protect: only logged-in users pass
exports.protect = async (req, res, next) => {
  const header = req.headers.authorization || '';
  // Expected:  Authorization: Bearer <token>
  if (!header.startsWith('Bearer '))
    return res.status(401).json({ message: 'Not authorized' });
  try {
    const decoded = jwt.verify(header.split(' ')[1],
                               process.env.JWT_SECRET);
    // attach the user (without password) for later handlers
    req.user = await User.findById(decoded.id).select('-password');
    next();
  } catch (err) {
    res.status(401).json({ message: 'Token invalid or expired' });
  }
};

// authorize: only the allowed roles pass
exports.authorize = (...roles) => (req, res, next) =>
  roles.includes(req.user.role)
    ? next()
    : res.status(403).json({ message: 'Forbidden' });
```

What it does

- protect: 401 when the token is missing, invalid or expired
- authorize('senior'): 403 for the wrong role
- Password is never attached to req.user

Role from year (server side)

```
// request body.role is ignored
role = year <= 2 ? 'junior' : 'senior';
```

Covers: AUTH-F3, F6, F7, N2

Code: Atomic Seat Registration

sessionService.register(userId, sessionId)

```
const updated = await Session.findOneAndUpdate(
  {
    _id: sessionId,
    mentorId: { $ne: userId },           // no self-registration
    registeredStudents: { $ne: userId }, // no duplicate
    startsAt: { $gt: new Date() },       // not started yet
    $expr: { $lt: [{ $size: '$registeredStudents' },
                   '$maxSeats'] }        // a seat is free
  },
  { $addToSet: { registeredStudents: userId } },
  { new: true }
);

if (updated)
  return { seatsLeft: updated.maxSeats - updated.registeredStudents.length };

// No match: read the session and explain why
const s = await Session.findById(sessionId);
if (!s) throw new AppError(404, 'Session not found');
if (String(s.mentorId) === String(userId))
  throw new AppError(400, 'Cannot join own session');
if (s.registeredStudents.includes(userId))
  throw new AppError(409, 'Already registered');
throw new AppError(409, 'Session is full');
```

Why one atomic update?

- Checks and update happen in one DB operation
- Two clicks on the last seat: one wins, one gets "Full"
- Seats left = maxSeats minus registered count

Error mapping

- 404 · 400 · 409 (already) · 409 (full)

Covers: SESS-F5 to F8, N1 · TC8

Code: Quiz Scoring & Points

quizService.submitAttempt() + pointsService.award()

```
finalizeIfExpired(attempt);   // late? status = AUTO_SUBMITTED

// Grade on the SERVER: the browser never sees correctIndex
let score = 0;
for (const q of quiz.questions) {
  const a = attempt.answers.find(x => x.questionId == q.id);
  if (a && a.selectedIndex === q.correctIndex) score += q.marks;
}
const percent = (score / quiz.totalMarks) * 100;
attempt.set({ score, percent, submittedAt: new Date() });
await attempt.save();

const points = 10 + Math.round(percent / 10);   // base + bonus
await award(userId, 'QUIZ_COMPLETE', quizId, points);

async function award(userId, action, refId, points) {
  try {
    // unique index (userId + action + refId) blocks double award
    await PointTransaction.create({ userId, action, refId, points });
  } catch (e) { if (e.code === 11000) return; throw e; }
  await User.findByIdAndUpdate(userId, { $inc: { points } });
}
```

Server-side scoring

- correctIndex is sent only after submit
- Late answers: deadline + 5 s is ignored

Worked example

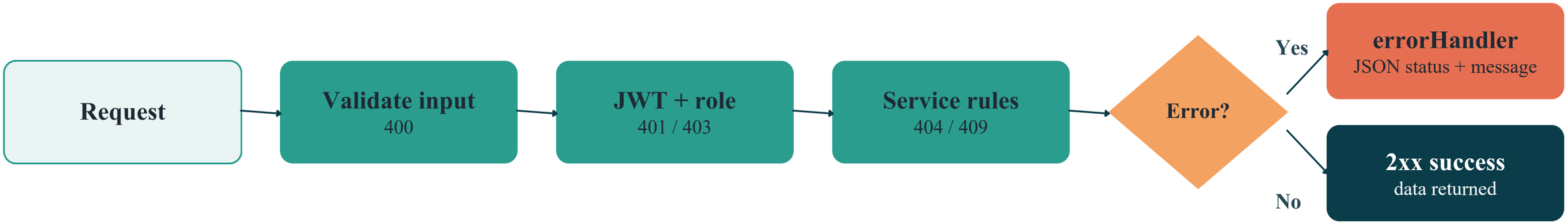
- 8 of 10 marks = 80%
- 10 + 8 = 18 points
- 18 + 5 attended + 2 rating = 25 points (Newcomer)

No double award · TC14

Exception Handling: Cases & Responses

Case	How it is caught	Response
Token missing or expired	JWT middleware (protect)	**401**
Wrong role, e.g. junior creates a session	authorize(role)	**403**
Edit/delete another mentor's session	Owner check on mentorId	**403**
Duplicate email at signup	Unique index on email	**409**
Wrong email or password	bcrypt compare, same message	**401**
Year outside 1 to 4	Body validation	**400**
Register twice, own session, or full	Atomic update result	**409 / 400**
Start quiz without registration	Registered check	**403**
Answer after deadline + 5 s	finalizeIfExpired()	**Ignored**
Rating outside 1-5 or invalid link	Range and http/https check	**400**
Database or server failure	Global error handler (details only in logs)	**500**

Exception Handling: Flow & Code



middleware/errorHandler.js

```
class AppError extends Error {
  constructor(status, message) { super(message); this.status = status; }
}
module.exports = (err, req, res, next) => {
  // Mongo duplicate key (e.g. same email) -> 409
  if (err.code === 11000)
    return res.status(409).json({ message: 'Already exists' });
  const status = err.status || 500;
  // never leak internals on a 500
  const message = status === 500 ? 'Server error' : err.message;
  if (status === 500) console.error(err);  // details only in logs
  res.status(status).json({ message });
};
```

Frontend handling

- Axios interceptor: 401 → logout() → /login
- Toast shows the server message
- EmptyState when nothing matches
- Broken resource link never crashes the page

Challenges Faced & How We Overcame Them

CHALLENGE	SOLUTION
Two students grabbing the last seat, or registering twice	One atomic findOneAndUpdate with $ne, $addToSet and $expr checks
Users could pick their own role and act as a senior	Server computes role from year; body.role is ignored
Quiz answers visible in the browser; timer reset on refresh	correctIndex stripped until submit; deadlineAt stored on server; finalizeIfExpired()
Points farmed by repeating actions	point_transactions with unique (userId, action, refId); one attempt per quiz
Team overwriting auth code; secrets leaking to GitHub	Separate Git branches, .env in .gitignore, shared protect/authorize reused

Commands & Team Workflow

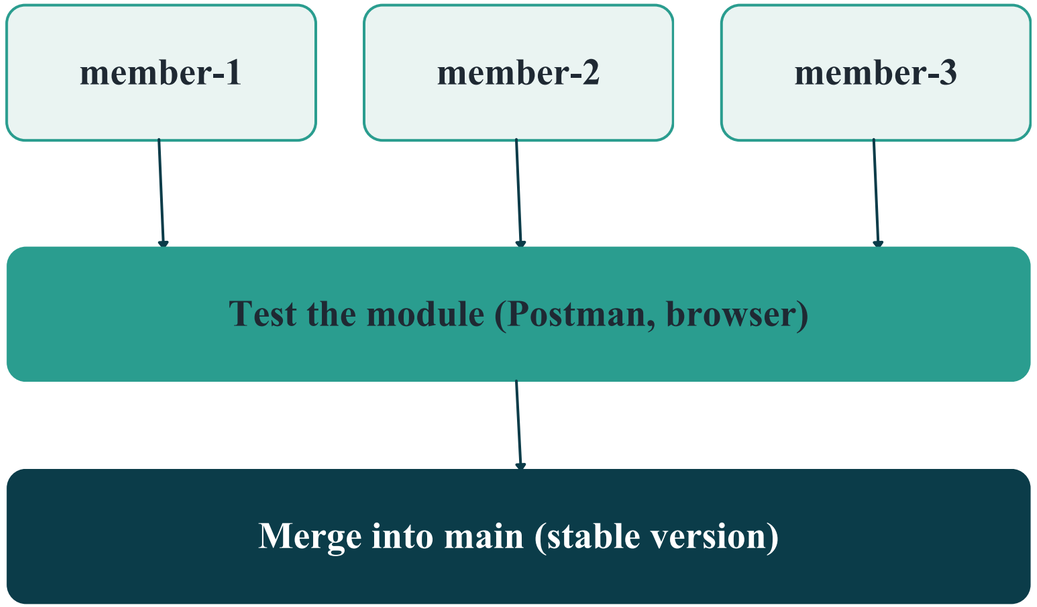
Setup and run commands

```
# 1. Clone and install
git clone <PeerBridge-repo-url>
cd PeerBridge/backend  && npm install
cd ../frontend         && npm install

# 2. Create backend/.env (never commit it)
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/peerbridge
JWT_SECRET=<team-agreed-value>
JWT_EXPIRES_IN=<duration>

# 3. Run both servers
cd backend  && npm run dev     # http://localhost:5000
cd frontend && npm run dev     # http://localhost:5173

# 4. Work on your own branch
git checkout -b member-2
git add . && git commit -m "message"
git push origin member-2
```



Team rules

- Never commit .env or node_modules
- Reuse the existing auth system
- One branch per module; test before merging

Status, Future Scope & Wrap-up

1 Authentication & Profile
Done

2 Session Management
In progress (Member 2)

3 Quiz
In progress (Member 3)

4 Leaderboard & Points
Planned

5 Ratings & Feedback
Planned

6 Resource Sharing
Planned

Future scope

- Admin panel
- AI doubt-clearing chatbot
- In-app notifications
- Deploy: Vercel + Render + MongoDB Atlas
- Real attendance tracking

Key takeaways

- Layered design keeps modules independent
- Server-side rules protect data and fairness
- Exceptions are handled in one central place

Thank you!

Questions?